



Instituto Superior de Engenharia

Politécnico de Coimbra

PROGRAMAÇÃO AVANÇADA

JOGO DE XADREZ

Autor / Author

José Moreira 2022132718

Miguel Realinho 2022157806

Gonçalo Oliveira 2022143112

Coimbra, Maio 2025



INSTITUTO
POLITÉCNICO DE
COIMBRA

ÍNDICES

Conteúdo

Índices	1
1 Introdução	2
2 Funcionalidades	3
2.1 Funcionalidades Estruturais e Arquitetura.....	3
2.2 Funcionalidades do modelo.....	4
2.3 Funcionalidades do UI	4
3 Classes e a sua função	5
4 Decisões de design	6
4.1 Peças	6
4.2 Tabuleiro	6
4.3 Implementação das regras.....	7
5 Padroes de programação	8
5.1 MVVM com facade.....	8
5.2 Memento	9
5.3 Padrão de Fábrica e de hierarquia	10

1 INTRODUÃO

No contexto deste trabalho, foi desenvolvido um jogo de xadrez em Java, com algumas funcionalidades de aprendizagem. Neste projeto foram aplicados vrios conceitos lecionados nas aulas. Entre esses conceitos, destaca-se a utilizao de vrios padres de programao e JavaFX.

Ao longo deste relatrio, explicaremos detalhes sobre a aplicao, escolhas feitas, diagramas, bem como os desafios encontrados e as solues propostas para os mesmos.

2 FUNCIONALIDADES

Todas as funcionalidades previstas no enunciado foram integralmente implementadas, de acordo com os requisitos de design estabelecidos ao longo do semestre.

2.1 Funcionalidades Estruturais e Arquitetura

Separação entre Modelo e UI	✓	Nenhuma lógica de jogo na UI, Interação da UI via Facade
Padrão arquitetural MVVM	✓	Separação clara entre modelo, vista e controlo
Padrão Memento	✓	Suporte a undo/redo via histórico de estados
Logger	✓	Registo de eventos e ações da aplicação
Padrão de notificações assíncronas	✓	São usados notificações assíncronas para atualizar toda a formação sobre o modelo nas views

2.2 Funcionalidades do modelo

Movimento das peas	✓	Cada pea devolve todos os seus movimentos possveis de acordo com o estado do tabuleiro
Gesto de turnos	✓	O turno de cada jogador  corretamente gerido, alterando entre os dois se o movimento for completado corretamente.
Captura de peas	✓	As peas so capturadas corretamente de acordo com as regras do jogo.
Movimentos especiais	✓	o Roque, en Passant e promoo do peo, so corretamente verificadas.
Verificao de checkmate/Stalemate	✓	Inclui regras completas para cada caso, verificando a possibilidade de bloqueio/captura/fuga
Validao de Movimentos	✓	Todas as peas tm os seus movimentos possveis validados, tendo tmbm em conta o rei no poder se colocar em perigo.

2.3 Funcionalidades do UI

Interface grfica para a aplicao	✓	Menu, com opes de iniciar um novo jogo/carregar um jogo/sair/etc.
Interface de jogo	✓	Visualizao de um tabuleiro de acordo com as dimenses provenientes do modelo de dados.
Feedback Sonoro	✓	Todas as jogadas devolvem um feedback sonoro da interao entre cada pea.
Visualizao de jogadas possveis	✓	Se ligado, mostra sucintamente as jogadas possveis da pea selecionada de acordo com o estado do tabuleiro.

3 CLASSES E A SUA FUNÇÃO

ChessMain	Classe main que inicializa a aplicação javafx
MainJFX	Classe que estende a Application, que configura o model e o UI
ChessGame	Classe que representa as regras de um tabuleiro, manipulando o tabuleiro, validando os possíveis movimentos, e guarda os dados necessários para tal
SaveLoadManager	Classe utilitária não instanciável responsável por guardar e carregar estados de jogo utilizando serialização
ChessGameMemento	Uma <i>nested class</i> dentro da ChessGame que implementa a interface IMemento, usada para implementar o padrão memento para o undo/redo de jogadas.
IMemento	Interface para a facade do ChessGame para encapsular as suas funções.
ChessGameManager	Facade do ChessGame, para implementar padrão MVVM, Sendo este o meio intermédio entre o Model e o View Model
PieceFactory	Classe utilitária para criar uma peça usando o padrão de factory, classe instanciável apenas usada na classe Board
Board	Classe que representa um tabuleiro de jogo, serve para guardar e manipular peças de jogo
Piece	Classe abstrata representativa de uma peça
Pawn/rook/ knight/king/queen	Classes que representam cada peça
ModelLog	Classe singleton para manter um log dos eventos do Model Data.

RootPane	Container principal para a aplicao que gere a interao com os menus, e informaes adicionais, como jogador atual
BoardPane	Container representativo do tabuleiro, gere as interaes do jogador com a facade
ImageManager	Classe que manipula as imagens, com cache
SoundManager	Classe que manipula os sons, tendo um mtodo para criar uma queue de sons.

4 DECISES DE DESIGN

4.1 Peas

Todas as peas derivam da classe “*Pie*”, com todas as propriedades encapsuladas como posio/cor, e mtodos para as manipular, incluindo um mtodo que devolve uma lista com as *strings* dos movimentos possveis de cada pea.

No caso dos movimentos possveis, todas as peas seguem os movimentos padro de um jogo de xadrez clssico, no entanto, ns decidimos fazer com que o mtodo para calcular estes devolve-se as posies sem validao, no caso do peo,  sempre devolvido as casas na sua diagonal da frente, sendo o “*ChessGame*” que depois valida a lista consoante o necessrio.

4.2 Tabuleiro

O tabuleiro de jogo  representado por uma classe no modelo de dados, com o propsito de guardar e manipular as peas mantendo conceitos de orientao de objetos por todo o modelo, sendo esta a nica classe que tem conhecimento das classes que representam cada tipo de pea.

Guardamos aqui um *ArrayList* com as peças todas que vão ser adicionadas ao tabuleiro, e disponibilizamos métodos para manipular as suas posições, a sua remoção.

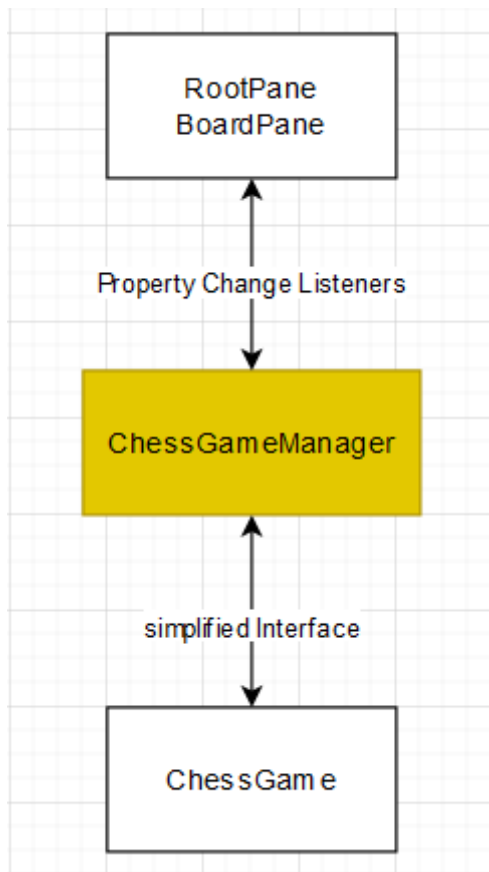
4.3 Implementação das regras

As regras de jogo são, na sua maior parte, implementadas pelo “*ChessGame*”. É aqui que os movimentos são filtrados. Certos movimentos como o roque e o “*en passant*” requerem ajuda adicional da “*board*”, sendo que a classe “*ChessGame*” não sabe da existência de tipos de peças, perguntando a “*board*” da posição do rei/ se a peça na posição x/y é um peão, etc.

Aqui também implementamos toda a logica de troca de jogadores, manipulada por um booleano “*whitetomove*” para nos ajudar a detetar qual o jogador que pode jogar, fazendo também bloquear a possibilidade de mover peças pretas na vez do jogador de Branco jogar.

5 PADRŐES DE PROGRAMAÃO COM AS RELAŐES ENTRE AS CLASSES

5.1 MVVM com “*facade*”



Representaão da separaão entre UI (*RootPane*, *BoardPane*), lógica de ligaão (*ChessGameManager*) e lógica do jogo (*ChessGame*) utilizando o padrão MVVM. A comunicaão entre os componentes é realizada via observadores e uma interface simplificada providenciada pela *facade*.

5.2 Memento

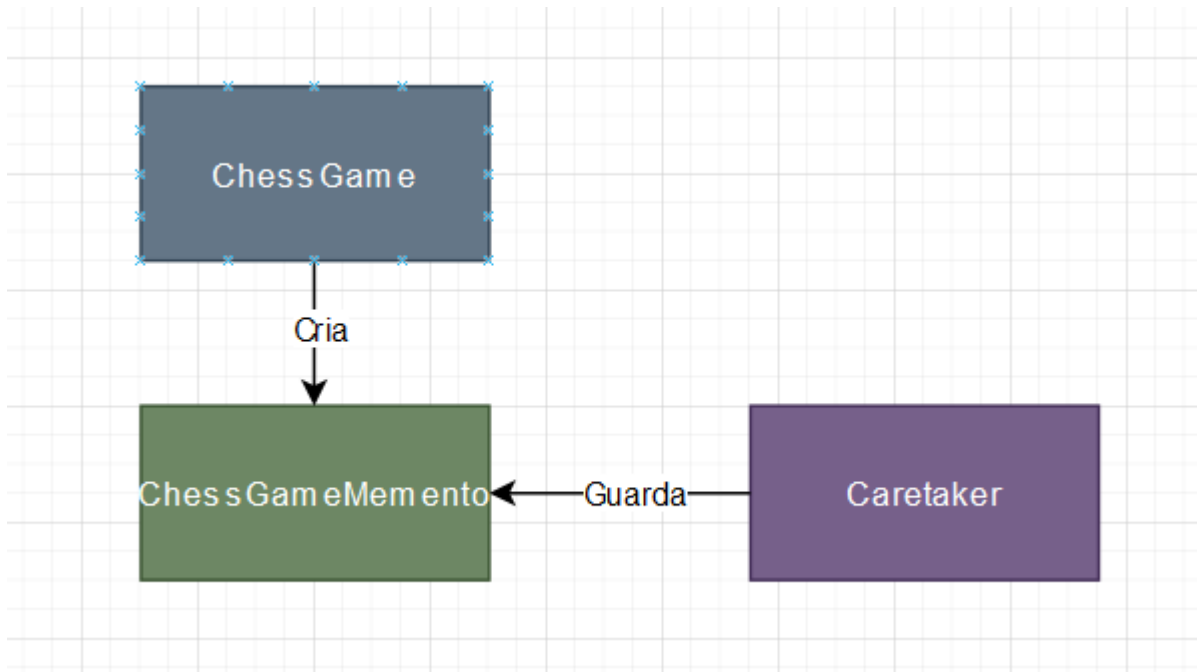
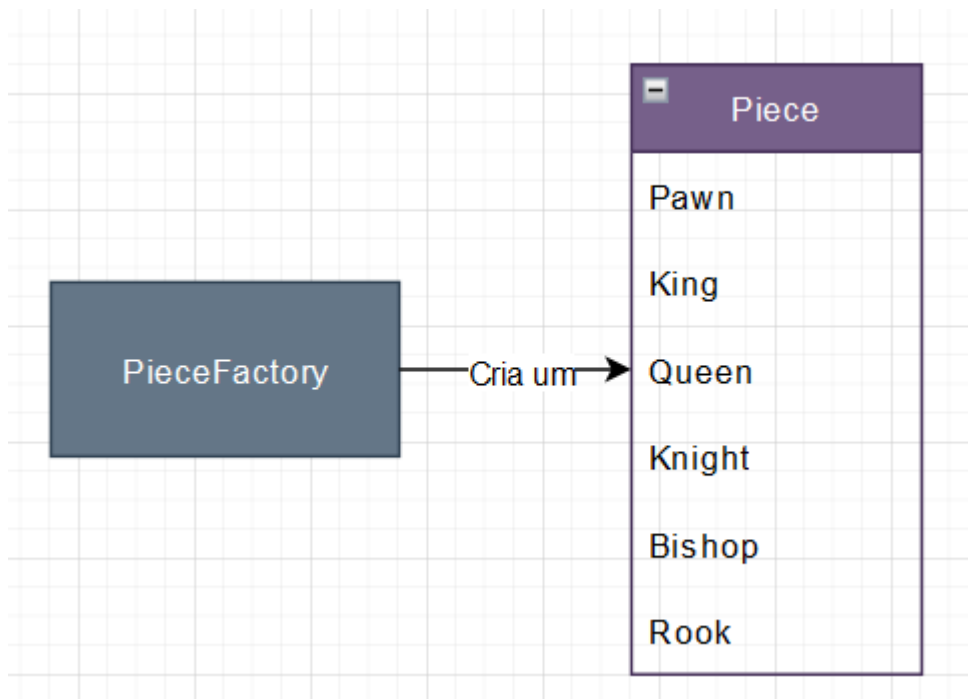


Ilustração do padrão Memento aplicado na classe “*ChessGame*”. Esta cria objetos do tipo “*ChessGameMemento*” que encapsulam o estado do jogo. Os mementos são armazenados por um “*caretaker*” para permitir funcionalidades como undo/redo

5.3 Padro de Fbrica e de hierarquia



Representao do padro *Factory* aplicado na criao de peas. A classe “*PieceFactory*” encapsula a lgica de criao de instncias concretas de “*Piece*”, como “*Pawn*”, “*King*”, “*Queen*”, etc., promovendo a flexibilidade e a centralizao da criao de objetos.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra